Security Audit Report

CoinList Token Sale Fund

v1.0

January 16, 2026

Table of Contents

Table of Contents 2
License 4
Disclaimer 5
Introduction 6
Purpose of This Report 6
Codebase Submitted for the Audit 6
Methodology 7
Functionality Overview 7
How to Read This Report 8
Code Quality Criteria 9
Summary of Findings 10
Detailed Findings 11
1. Lack of minimum commit/remit enforcement enables economic DoS attack on operators 11
2. Zero-value commits and remits inflate counters and emit misleading events 11
3. Owner can drain all funds regardless of outstanding user commitments 12
4. Users must trust the COMMIT_LEVEL role to commit the intended amount and option 13
5. A single blocklisted user can block an entire remit batch transaction 13
6. Centralized remitter role enables remit request censorship 14
7. Add a helper view function for calculating the user commitment balance 14
8. Missing token whitelisting allows arbitrary asset commitments and breaks assumptions 15
9. Redundant pre-transfer allowance and balance checks can be removed 15
10. Unbounded commit/remit operations may lead to integer overflow and DoS 16
11. Unpinned Solidity compiler version may lead to inconsistent bytecode across builds 16
Security Model 18
Assets 18
ERC-20 tokens 18
ERC-20 allowances 18
Private keys controlling external accounts 18
Gas price used by operators for protocol functionality 19
Stakeholders/Potential Threat Actors 20
Users 20
CoinList backend systems 20
Owner operators 20
Committer role operators 20
Remitter role operators 20

Sale partner 20
ERC20 token contracts 20
Supply chain 20
Attackers 21
Assumptions 22
Token assumptions 22
Privilege and operations assumptions 22
Threat Model 23
Process Applied 23
STRIDE Interpretation in the Blockchain Context 23
STRIDE Classification 25
Mitigation Matrix 26
TokenSaleFund 26
Backend Systems 28
ERC20 tokens 30
Externally owned account 32

License

THIS WORK IS LICENSED UNDER A CREATIVE COMMONS ATTRIBUTION-NODERIVATIVES 4.0 INTERNATIONAL LICENSE.

Disclaimer

THE CONTENT OF THIS AUDIT REPORT IS PROVIDED “AS IS”, WITHOUT REPRESENTATIONS AND WARRANTIES OF ANY KIND.

THE AUTHOR AND HIS EMPLOYER DISCLAIM ANY LIABILITY FOR DAMAGE ARISING OUT OF, OR IN CONNECTION WITH, THIS AUDIT REPORT.

THIS AUDIT REPORT WAS PREPARED EXCLUSIVELY FOR AND IN THE INTEREST OF THE CLIENT AND SHALL NOT CONSTRUE ANY LEGAL RELATIONSHIP TOWARDS THIRD PARTIES. IN PARTICULAR, THE AUTHOR AND HIS EMPLOYER UNDERTAKE NO LIABILITY OR RESPONSIBILITY TOWARDS THIRD PARTIES AND PROVIDE NO WARRANTIES REGARDING THE FACTUAL ACCURACY OR COMPLETENESS OF THE AUDIT REPORT.

FOR THE AVOIDANCE OF DOUBT, NOTHING CONTAINED IN THIS AUDIT REPORT SHALL BE CONSTRUED TO IMPOSE ADDITIONAL OBLIGATIONS ON COMPANY, INCLUDING WITHOUT LIMITATION WARRANTIES OR LIABILITIES.

COPYRIGHT OF THIS REPORT REMAINS WITH THE AUTHOR.

This audit has been performed by

Oak Security GmbH

https://oaksecurity.io/
info@oaksecurity.io

Introduction

Purpose of This Report

Oak Security GmbH has been engaged by Amalgamated Token Services Inc. to perform a security audit of CoinList Token Sale Fund.

The objectives of the audit are as follows:

1. Determine the correct functioning of the protocol, in accordance with the project specification.
2. Determine possible vulnerabilities, which could be exploited by an attacker.
3. Determine smart contract bugs, which might lead to unexpected behavior.
4. Analyze whether best practices have been applied during development.
5. Make recommendations to improve code safety and readability.

This report represents a summary of the findings.

As with any code audit, there is a limit to which vulnerabilities can be found, and unexpected execution paths may still be possible. The author of this report does not guarantee complete coverage (see disclaimer).

Codebase Submitted for the Audit

The audit has been performed on the following target:

Repository	https://github.com/coinlist/solidity
Commit	`136f9040f0a3423e1f870d07cde4f7f913cfccee`
Scope	The scope is restricted to the `src/sale/TokenSaleFund.sol` contract.
Fixes verified at commit	`4314df154e9865c112224f5ee9fc189dc63de918` Note that only fixes to the issues described in this report have been reviewed at this commit. Any further changes such as additional features have not been reviewed.

Methodology

The audit has been performed in the following steps:

1. Gaining an understanding of the code base's intended purpose by reading the available documentation.
2. Automated source code and dependency analysis.
3. Manual line-by-line analysis of the source code for security vulnerabilities and use of best practice guidelines, including but not limited to:
    a. Race condition analysis
    b. Under-/overflow issues
    c. Key management vulnerabilities
4. Report preparation

Functionality Overview

TokenSaleFund is a custodial contract that temporarily holds users' contributed funds for a token sale and provides controlled flows for locking, refunding, and finally distributing those funds.

Designated backend-controlled accounts can move funds from a user into the contract ("commit"), which increases that user's recorded contribution and locks the corresponding tokens in the contract.

Other designated accounts can later send some or all of a user's locked funds back to them using remit.

After all intended refunds have been processed, the contract owner can transfer the remaining token balances held by the contract to a specified recipient, such as the sale partner.

How to Read This Report

This report classifies the issues found into the following severity categories:

Severity	Description
Critical	A serious and exploitable vulnerability that can lead to loss of funds, unrecoverable locked funds, or catastrophic denial of service.
Major	A vulnerability or bug that can affect the correct functioning of the system, lead to incorrect states or denial of service.
Minor	A violation of common best practices or incorrect usage of primitives, which may not currently have a major impact on security, but may do so in the future or introduce inefficiencies.
Informational	Comments and recommendations of design decisions or potential optimizations, that are not relevant to security. Their application may improve aspects, such as user experience or readability, but is not strictly necessary. This category may also include opinionated recommendations that the project team might not share.

The status of an issue can be one of the following: **Pending**, **Acknowledged**, **Partially Resolved**, or **Resolved**.

Note that audits are an important step to improving the security of smart contracts and can find many issues. However, auditing complex codebases has its limits and a remaining risk is present (see disclaimer).

Users of the system should exercise caution. In order to help with the evaluation of the remaining risk, we provide a measure of the following key indicators: **code complexity**, **code readability**, **level of documentation**, and **test coverage**. We include a table with these criteria below.

Note that high complexity or low test coverage does not necessarily equate to a higher risk, although certain bugs are more easily detected in unit testing than in a security audit and vice versa.

Code Quality Criteria

The auditor team assesses the codebase's code quality criteria as follows:

Criteria	Status	Comment
Code complexity	**Medium-Low**	-
Code readability and clarity	**Medium-High**	-
Level of documentation	**Medium-High**	The documentation clearly defines the contract.
Test coverage	**High**	Test coverage reported by `forge coverage` is `96.83%`

Summary of Findings

No	Description	Severity	Status
1	Lack of minimum commit/remit enforcement enables economic DoS attack on operators	Minor	Resolved
2	Zero-value commits and remits inflate counters and emit misleading events	Minor	Resolved
3	Owner can drain all funds regardless of outstanding user commitments	Minor	Acknowledged
4	Users must trust the `COMMIT_LEVEL` role to commit the intended `amount` and `option`	Minor	Acknowledged
5	A single blocklisted user can block an entire `remit` batch transaction	Minor	Acknowledged
6	Centralized remitter role enables remit request censorship	Minor	Acknowledged
7	Add a helper view function for calculating the user commitment balance	Informational	Resolved
8	Missing token whitelisting allows arbitrary asset commitments and breaks assumptions	Informational	Acknowledged
9	Redundant pre-transfer allowance and balance checks can be removed	Informational	Resolved
10	Unbounded commit/remit operations may lead to integer overflow and DoS	Informational	Resolved
11	Unpinned Solidity compiler version may lead to inconsistent bytecode across builds	Informational	Resolved

Detailed Findings

1. Lack of minimum commit/remit enforcement enables economic DoS attack on operators

Severity: Minor

The TokenSaleFund contract does not impose any minimum thresholds for commit or remit operations, opening a vector for a gas-based denial of service attack.

If off-chain components (such as UI-triggered backends) are configured to automatically execute on-chain commit and remit transactions on behalf of users with privileged roles (`COMMIT_LEVEL` and `REMIT_LEVEL`), a malicious user can exploit this by issuing minimal-value transactions (e.g., `1 wei`) in rapid succession.

These repeated cycles return the user to an equivalent token position while forcing the operator-controlled commit and remit accounts to expend gas repeatedly.

Consequently, this lack of rate limiting or minimum value enforcement exposes the system to significant economic inefficiency, potentially depleting operator budgets and impairing service availability.

Recommendation

We recommend enforcing on-chain minimum thresholds for `commit` and `remit` operations to reject transactions with negligible value. Additionally, rate-limiting mechanisms should be implemented on the off-chain components to prevent repeated invocation of privileged operations within short timeframes.

Status: Resolved

2. Zero-value commits and remits inflate counters and emit misleading events

Severity: Minor

In `src/sale/TokenSaleFund.sol:46-72`, the `commit` function accepts a zero `amount` parameter without restriction. All invariants pass (allowance and balance checks succeed trivially for zero), the `Committed` event is emitted with a zero amount, and `commitCount` is incremented while `commitSum` remains unchanged.

Similarly, in lines `74-98` and `100-139`, both `remit` function variants accept zero amounts. The commitment check `comm >= amount` passes, counters are incremented, and `Remitted` events are emitted.

This leads to the following consequences:

1. Commit and remit counters become skewed, as they do not correspond to actual economic activity
2. Zero-amount events may mislead off-chain consumers relying on event data
3. Zero-value ERC20 transfers may revert on non-standard tokens, causing unexpected failures for otherwise valid calls

Recommendation

We recommend adding a check to reject zero-value amounts in both `commit` and `remit` functions.

Status: Resolved

3. Owner can drain all funds regardless of outstanding user commitments

Severity: Minor

In `src/sale/TokenSaleFund.sol:144-155`, the `transfer` function allows the contract owner to transfer the entire token balance to any address. The function only verifies that the balance is greater than zero before executing the transfer.

However, the function does not account for users' outstanding commitment balances. The owner can withdraw all tokens at any time, even when users have committed funds that have not yet been remitted. This leaves affected users unable to reclaim their committed funds.

We classify this issue as minor since it requires a compromised or malicious owner to exploit.

Recommendation

We recommend implementing a safeguard that prevents the owner from withdrawing more than the difference between the contract's token balance and the total outstanding user commitments.

Alternatively, consider adding a timelock mechanism for withdrawals, allowing users to exit before funds are moved.

Status: Acknowledged

4. Users must trust the `COMMIT_LEVEL` role to commit the intended `amount` and `option`

Severity: Minor

In `src/sale/TokenSaleFund.sol:46-72`, the `commit` function transfers funds from a user to the contract. This function is not called directly by users. Instead, it is invoked by an address with the `COMMIT_LEVEL` role on behalf of users.

The function only validates that the user's allowance is at least as large as the specified `amount`. However, if a user has approved an allowance larger than the intended commit amount, the `COMMIT_LEVEL` role can commit up to the entire allowance in a single call or across multiple calls.

Additionally, the user has no on-chain control over which `option` is associated with their commitment, as the parameter is set solely by the caller.

This design introduces the following trust assumptions:

1. Users must trust that the `COMMIT_LEVEL` role commits only the intended amount and does not exceed its allowance.
2. Users must trust that the caller specifies the correct option.

Recommendation

We recommend clearly documenting these trust assumptions. Furthermore, we recommend that the frontend create approval transactions only for the specific commit amount, rather than using infinite allowances, to minimize the trust surface.

Status: Acknowledged

5. A single blocklisted user can block an entire `remit` batch transaction

Severity: Minor

In `src/sale/TokenSaleFund.sol:100-139`, the permissioned `remit` function iterates over an array of users and transfers tokens to each of them. If any single transfer fails, for example, when a user is blocklisted by a token like USDC, the entire batch transaction reverts.

This behavior can cause issues depending on how the off-chain script handles failed transactions. If the script simply retries a failed batch, it will not proceed because the blocklisted user will repeatedly cause the transaction to revert.

Recommendation

We recommend implementing a mechanism to skip failed transfers instead of reverting the entire batch. Failed transfers should be emitted as part of an event to enable off-chain tracking and manual resolution.

Status: Acknowledged

6. Centralized remitter role enables remit request censorship

Severity: Minor

The `remit` function in TokenSaleFund requires execution by an account with the `REMIT_LEVEL` role.

This centralized privilege model creates a single point of failure where a malicious or non-cooperative remitter can censor remit requests by refusing to execute them. Users cannot unilaterally trigger remittance, even if they have sufficient commitments.

This undermines decentralization and introduces a censorship risk where remittances can be indefinitely delayed or withheld at the discretion of a single role holder, effectively locking user funds within the contract.

Recommendation

We recommend allowing users to trigger their own remits within safe bounds or introducing automated mechanisms for remittance fulfillment.

Status: Acknowledged

7. Add a helper view function for calculating the user commitment balance

Severity: Informational

In `src/sale/TokenSaleFund.sol`, the `TokenSaleFund` contract provides a `fundingTokenBalance` view function for querying the contract's token balance.

However, the previous version of the contract also included a `commitBalance` helper function that allowed users and integrators to easily query a user's net commitment balance.

The new version stores commitment and remit data in the nested `totals` mapping with the structure `totals[user][option][token]`, which returns a `Total` struct containing `commitSum` and `remitSum` values. Without the helper function, users and integrators must manually query the `totals` mapping and calculate `commitSum - remitSum` to determine the effective commitment balance.

This creates unnecessary friction for off-chain integrations and increases the complexity of querying commitment state, potentially leading to integration errors.

Recommendation

We recommend adding a `commitBalance` helper view function that accepts `user`, `option`, and `token` parameters and returns the calculated net commitment balance (`commitSum - remitSum`).

Status: Resolved

8. Missing token whitelisting allows arbitrary asset commitments and breaks assumptions

Severity: Informational

The `commit` function in TokenSaleFund permits the `COMMIT_LEVEL` role to commit any ERC-20 token on behalf of a user.

Although the system is intended to support only stablecoins like USDT and USDC, this restriction is not enforced in the contract logic.

As a result, committers can interact with any token, including those with non-standard behaviors such as fee-on-transfer mechanisms. This lack of validation can break balance-based invariants (e.g., assuming `transferFrom` moves exactly the committed amount), potentially causing inconsistencies.

Recommendation

We recommend implementing an explicit whitelist of allowed token addresses within the constructor or via a governance-controlled mechanism.

Status: Acknowledged

9. Redundant pre-transfer allowance and balance checks can be removed

Severity: Informational

In `src/sale/TokenSaleFund.sol:54-59`, the `commit` function checks both the user's allowance and balance before calling `trySafeTransferFrom`. These checks are redundant because the underlying ERC-20 `transferFrom` call will revert if either the allowance or balance is insufficient.

Similar redundant balance checks exist in:

1. `src/sale/TokenSaleFund.sol:87-88` in the single `remit` function
2. `src/sale/TokenSaleFund.sol:147-148` in the `transfer` function

These additional checks increase gas costs without providing additional safety guarantees.

Recommendation

We recommend removing the redundant allowance and balance checks.

Status: Resolved

10. Unbounded commit/remit operations may lead to integer overflow and DoS

Severity: Informational

The `commit` and `remit` functions in the `TokenSaleFund` contract increment the `committed` and `remitted` mappings without any upper bounds or sanity checks.

Since there is no fee imposed on the user and no limit on the frequency or volume of these operations, a malicious actor could continuously trigger commits and remits with negligible amounts (e.g., `1 wei`).

Over time, this may cause an overflow in the per-user or total counters, leading to undefined behavior or system failure.

Recommendation

We recommend introducing minimum value thresholds for `commit` and `remit` operations and adding usage fees or rate limiting to reduce the incentive and feasibility of abuse.

Status: Resolved

11. Unpinned Solidity compiler version may lead to inconsistent bytecode across builds

Severity: Informational

The `TokenSaleFund` contract declares `pragma solidity ^0.8.27`, allowing any compiler version from `0.8.27` up to, but not including `0.9.0`. This floating pragma allows different environments to compile contracts with different patch-level compilers, each potentially emitting slightly different bytecode or enabling/disabling compiler-level bug fixes.

Such variability complicates deterministic builds, auditing, and reproducibility of deployed code, and could introduce latent bugs if future compiler releases change semantics or optimize code differently. Without a pinned compiler, audits performed against one binary may not match the bytecode actually deployed later.

Recommendation

We recommend pinning the compiler to an exact version (e.g., `0.8.27`).

Status: Resolved

Security Model

The security model has been carefully crafted to delineate the various assets, actors, and underlying assumptions of the TokenSaleFund contract. They will then be analyzed in a threat model to outline high-level security risks and proposed mitigations.

The purpose of this security model is to recognize and assess potential threats, as well as the derivation of recommendations for mitigations and counter-measures.

There is a limit to which security risks can be identified by constructing a security/threat model. Some risks may remain undetected and may not be covered in model described below (see disclaimer).

Assets

The following outlines assets that hold significant value to potential attackers or other stakeholders of the system.

ERC-20 tokens

The primary asset is the ERC20 token balance, which is custodying user funds during the sale funding step.

These balances must be protected against unauthorized withdrawal, loss of availability, and insolvency relative to internal accounting.

ERC-20 allowances

ERC20 allowances represent the permissions users grant to the TokenSaleFund contract to transfer tokens from their wallet via `transferFrom`. In this system, allowances are a critical asset because the `commit(user, token, amount)` flow is executed by privileged CoinList backend accounts and relies on the user having previously approved the contract.

Allowances are particularly attractive to attackers because they can be abused without requiring the user to sign additional transactions at the time of abuse.

Private keys controlling external accounts

Regarding externally owned accounts, a specific entity retains control over the private key associated with the administration accounts. The significance of this private key lies in granting full authority over the account, making it an attractive target for attackers.

Access to the private key bestows the attacker with the ability to impersonate the legitimate account owner and execute actions from that account.

Gas price used by operators for protocol functionality

If the gas price gets prohibitively expensive, it may lead to a circumstance where the protocol or its regular operation (e.g., execution of commitment) becomes economically non-viable or impractical from a business perspective.

Stakeholders/Potential Threat Actors

The following outlines the various stakeholders or potential threat actors that interact with the system.

Users

Participants approve token allowances and expect funds to be pulled only when they are eligible and only for intended amounts.

Users are vulnerable to phishing, malicious UI flows, and allowance misuse.

CoinList backend systems

Backend services perform due diligence, decide qualification, and orchestrate calls using privileged roles. They are a major trust anchor and a major attack surface.

Owner operators

Critical operators controlling the owner account, expected to be multisig or enclave backed.

Owner compromise is catastrophic.

Committer role operators

Backend controlled accounts that can call `commit(user, token, amount)` and pull approved user funds.

Remitter role operators

Backend controlled accounts that can call `remit` functions and return funds to users.

Sale partner

Receives net raised funds after refunds. Incorrect recipient selection is catastrophic.

ERC20 token contracts

External token contracts are outside the system trust boundary. Non standard behavior, pausing, blacklisting, fee on transfer mechanics, and reentrancy capable tokens can impact safety and liveness.

Supply chain

The contract depends on Solady (`OwnableRoles`, `SafeTransferLib`) and the compiler toolchain. A compromised dependency can subvert authorization or transfer semantics.

Attackers

External malicious actors seeking to:

- Steal committed funds
- Manipulate accounting state
- Disrupt sale operations (DoS)
- Exploit token approval mechanics

Assumptions

The following outlines various assumptions upon which the system's functioning is predicated.

Token assumptions

- Only known stablecoins that strictly adhere to the ERC-20 standard are used as funding sources in production, including USDC and USDT.
- No fee-on-transfer, rebasing, or otherwise deflationary mechanics are used for funding tokens.
- Token contracts do not implement behaviors that can break transfers to or from the fund contract, such as blacklisting the contract, pausing, or forcing unexpected reverts.

Privilege and operations assumptions

- Owner key management is robust, multisig and or enclave backed with strict access controls.
- Committer and remitter accounts are hardened, monitored, and separated by least privilege.
- Off-chain due diligence and eligibility checks are correct and enforced.
- Backend uses strict allowlists for supported token addresses per sale.

Threat Model

Process Applied

The process performed to analyze the system for potential threats and build a comprehensive model is based on the approach first pioneered by Microsoft in 1999 that has developed into the STRIDE model

(https://docs.microsoft.com/en-us/previous-versions/commerce-server/ee823878(v=cs.20).

Whilst STRIDE is aimed at traditional software systems, it is generic enough to provide a threat classification suitable for blockchain applications with little adaptation (see below).

The result of the STRIDE classification has then been applied to a risk management matrix with simple countermeasures and mitigations suitable for blockchain applications.

STRIDE Interpretation in the Blockchain Context

STRIDE was first designed for closed software applications in permissioned environments with limited network capabilities. However, the classification provided can be adapted to blockchain systems with small adaptations. The below table highlights a blockchain-centric interpretation of the STRIDE classification:

Spoofing	In a blockchain context, the authenticity of communications is built into the underlying cryptographic public key infrastructure. However, spoofing attack vectors can occur at the off-chain level and within a social engineering paradigm. An example of the former is a Sybil attack where an actor uses multiple cryptographic entities to manipulate a system (wash-trading, auction smart contract manipulation, etc.). The latter usually consists of attackers imitating well-known actors, for instance, the creation of an impersonation token smart contract with a malicious implementation.
Tampering	Similarly to spoofing, tampering of data is usually not directly relevant to blockchain data itself due to cryptographic integrity. It can still occur though, for example through compromised developers of the protocol that have access to deployment keys or through supply chain attacks that manages to inject malicious code or substitutes trusted software that interacts with the blockchain (node software, wallets, libraries).

Repudiation	Repudiation, i.e. the ability of an actor to deny that they have taken action is usually not relevant at the transaction level of blockchains. However, it makes sense to maintain this category, since it may apply to additional software used in blockchain applications, such as user-facing web services. An example is the claim of a loss of a private key and hence assets.
Information Disclosure	Information disclosure has to be treated differently at the blockchain layer and the off-chain layer. Since the blockchain state is inherently public in most systems, information leakage here relates to data that is discoverable on the blockchain, even if it should be protected. Predictable random number generation could be classified as such, in addition to simply storing private data on the blockchain. In some cases, information in the mempool (pending/unconfirmed transactions) can be exploited in front-running or sandwich attacks. At the off-chain layer, the leakage of private keys is a good example of operational threat vectors.
Denial of Service	Denial of service threat vectors translates directly to blockchain systems at the infrastructure level. At the smart contract or protocol layer, there are more subtle DoS threats, such as unbounded iterations over data structures that could be exploited to make certain transactions not executable.
Elevated Privileges	Elevated privilege attack vectors directly translate to blockchain services. Faulty authorization at the smart contract level is an example where users might obtain access to functionality that should not be accessible.

STRIDE Classification

The following threat vectors have been identified using the STRIDE classification, grouped by components of the system.

	Spoofing	**Tampering**	**Repudiation**	**Information Disclosure**	**Denial of Service**	**Elevated Privileges**
TokenSale Fund	-	-	Commit/remit intent disputes	-	-	Role misassignment, owner overreach
Backend systems	Phished or compromised operators	-	Operator denial of actions	-	Backend system unavailability	-
ERC20 tokens	Spoofed token addresses	Fee-on-transfer, pausing, blacklisting	-	-	Transfer blocking	-
Externally owned account	Lost account	Pharming/ phishing/ social engineering	Compromised account	Private key leakage Doxxing/identity disclosure	DOS of infrastructure	Compromised private key

Mitigation Matrix

The following mitigation matrix describes each of the threat vectors identified in the STRIDE classification above, assigning an impact and likelihood and suggesting countermeasures and mitigation strategies. Countermeasures can be taken to identify and react to a threat, while mitigation strategies prevent a threat or reduce its impact or likelihood.

TokenSaleFund

Threat Vector	Impact	Likelihood	Mitigation	Countermeasures
Commit/remit intent disputes User disputes that they authorized a commit, claiming CoinList committed without consent or committed the wrong amount. Similarly, a user disputes the remit amount, claiming they were entitled to more or received the wrong token.	Medium	Medium	Emit detailed events with user, token, and amount, require explicit user approval via UI before backend commits, store off-chain records linking user approval to on-chain commit.	Maintain comprehensive audit trail correlating UI actions to on-chain events, provide users with transaction receipts, enable users to verify commits via block explorer.
Role misassignment, owner overreach The owner grants `COMMIT_LEVEL` or `REMIT_LEVEL` to unauthorized or malicious addresses, enabling unauthorized fund movements. Owner calls the `transfer` function prematurely before all users have been remitted, resulting in users being unable to recover their funds.	Critical	Low	Implement strict role-assignment procedures, use multisig for the owner role, and verify role assignments after granting. Establish clear operational procedures for when the `transfer` function is permitted, require off-chain verification that all remits are complete	Monitor and audit emitted events. Use an alerting system to notice role changes.

Threat Vector	Impact	Likelihood	Mitigation	Countermeasures
			before transfer	

Backend Systems

Threat Vector	Impact	Likelihood	Mitigation	Countermeasures
Phished or compromised operators An attacker compromises operator credentials through phishing, social engineering, or system breach, gaining ability to execute commit or remit operations.	High	Medium	Use hardware security modules (HSM) or secure enclaves (Nitro) for key storage, implement multi-party approval for sensitive operations, enforce strict access controls to backend systems, require MFA for all operator access, and conduct regular security training for operators.	Monitor for unusual transaction patterns (off-hours activity, abnormal amounts, unexpected frequency), implement limits on commit/remit operations, maintain audit logs, have incident response plan with key rotation procedures, use separate keys for committer and remitter roles.
Operator denial of actions An operator denies having performed a commit or remit action, either to avoid accountability for an error or to cover malicious activity	Medium	Low	Maintain audit logs correlating operator identity to on-chain transactions, store logs in immutable/append-only storage.	Enforce clear accountability policies, use separate logging systems that operators cannot modify, and have clear escalation and investigation procedures for disputed

Threat Vector	Impact	Likelihood	Mitigation	Countermeasures
				actions.
Backend system unavailability The backend system becomes unavailable due to infrastructure failure, DDoS attack, or operational error, preventing commit or remit operations from being processed during an active sale.	Medium	Medium	Deploy backend systems with high availability architecture (redundancy, failover), use geographically distributed infrastructure, implement health monitoring and auto-recovery, maintain backup signing infrastructure, and establish SLAs for system uptime during active sales.	Monitor system health, have on-call procedures for rapid incident response, maintain runbooks for common failure scenarios, and communicate expected timelines to users during outages.

ERC20 tokens

Threat Vector	Impact	Likelihood	Mitigation	Countermeasures
Spoofed token addresses An attacker provides a malicious token contract address that mimics a legitimate stablecoin but contains backdoors, reentrancy vectors, or manipulated return values.	Medium	Low	Maintain an explicit whitelist of approved funding token addresses, verify token contract addresses against official sources before adding to the whitelist, validate token addresses in backend systems before passing to contract functions, and use checksummed addresses throughout all systems.	Monitor for commit/remit attempts with non-whitelisted tokens, alert on any transaction involving unknown token addresses, verify token contract bytecode/source matches expected implementation, and cross-reference token addresses with multiple authoritative sources.
Fee-on-transfer, pausing, blacklisting The token contract has non-standard behavior: it deducts fees during transfer (breaking accounting), can be paused by the issuer (blocking all operations), or can blacklist addresses (preventing specific transfers).	High	Low	Only support well-known standard token implementations (USDC, USDD), verify token behavior through testing before deployment, and check the token contract for pausability and admin functions during due diligence.	Monitor contract balance vs sum of (committed - remitted) for discrepancies indicating fee-on-transfer, alert if token transfer fails unexpectedly, track token issuer announcement

Threat Vector	Impact	Likelihood	Mitigation	Countermeasures
				s regarding contract upgrades or blacklist actions, and have contingency procedures for handling users affected by token-level restrictions.
Transfer blocking Token issuer (e.g., Circle for USDC, Tether for USDT) blocks transfers to/from the contract address or specific user addresses, preventing commits, remits, or transfers from completing.	High	Low	Monitor blocklist status of contract addresses and high-value user addresses.	Alert immediately if contract address or user addresses appear on blocklists, have escalation procedures to token issuer for blocklist disputes, document affected users and amounts for potential recovery, communicate transparently with affected users about the situation and remediation options.

Externally owned account

Threat Vector	Impact	Likelihood	Mitigation	Countermeasures
Lost account The attacker claims that they own an account and the access to the private key has been lost	Low	Low	Have a clear policy not to refund lost assets or restore privileges	Enforce policy and strictness
Pharming/phishing/social engineering The attacker may manipulate users' or development teams' wallets, lure them to malicious front-ends, manipulate DNS records, or use social engineering to trick users/teams into signing manipulated transactions transferring funds/permissions	Medium	Medium	Educate users and team, protect DNS records, create awareness, offer blacklists with malicious sites, create activity on social channels to build reputable channels, deploy front-ends on IPFS or other decentralized infrastructure	Monitor all systems, monitor communities and impersonations/malicious copies of official channels, communicate attempted pharming/phishing/social engineering, have processes in place to recover from DNS manipulation, attacks on front-ends quickly
Compromised account The attacker claims they are a victim of scapegoating, denying responsibility for their attack	Low	Low	Have a clear policy not to refund lost assets or restore privileges	Enforce policy
Private key leakage Private keys are accidentally shared or logged	Medium	Medium	Educate users and team, ensure private keys are properly handled in wallet software, use	Monitor all systems, have policy in place to rotate keys

Threat Vector	Impact	Likelihood	Mitigation	Countermeasures
			hardware wallets/air-gapped devices, security keys, multi-signatures	
Doxxing/identity disclosure Private data such as the off-chain identity of users disclosed	Low	Medium	Educate users and team, no storage of identity/sensible data in databases that link identity to account addresses, follow privacy regulations and guidelines	-
DOS of infrastructure DOS attack on a validator, relayer, an end user's device/network or on the blockchain node they interact with	Low	Low	Educate users and team, use firewalls, sentry architecture, load balancers, VPNs	Monitor infrastructure, and have processes in place to elastically provision and deploy additional resources
Compromised private key Private keys may be compromised	Medium	Medium	Educate users and team	Monitor all systems, have policy in place to rotate keys